Experiment 1

Program to convert hexadecimal, decimal number to binary number

---

```
#include <stdio.h> #include <string.h>

void decimalToBinary(int decNum)
{
int binary[32]; int i = 0;
if (decNum == 0)
{
printf("0"); return;
}

while (decNum > 0)
{
binary[i] = decNum % 2; decNum = decNum / 2; i++;
}

while (i > 0)
{
printf("%d", binary[--i]);
}
}

void hexToBinary(char hexNum[])
{
int i;

for (i = 0; hexNum[i] != '\0'; i++)
{
switch (hexNum[i])
{
case '0': printf("0000"); break;
case '1': printf("0001"); break;
case '2': printf("0010"); break;
case '3': printf("0011"); break;
case '4': printf("0100"); break;
case '5': printf("0101"); break;
            case '6': printf("0110"); break;
case '7': printf("0111"); break;
case '8': printf("1000"); break;
case '9': printf("1001"); break; case 'A':
case 'a': printf("1010"); break; case 'B':
case 'b': printf("1011"); break; case 'C':
case 'c': printf("1100"); break; case 'D':
case 'd': printf("1101"); break; case 'E':
case 'e': printf("1110"); break; case 'F':
case 'f': printf("1111"); break; default:
printf("Invalid hexadecimal number"); return;
}
}
```

Experiment 1 (continued)

```
}

int main()
{
int decNum;
char hexNum[20];

printf("Enter a Decimal number: "); scanf("%d", &decNum);
printf("Binary equivalent: "); decimalToBinary(decNum);
printf("\n\n");

printf("Enter a Hexadecimal number: "); scanf("%s", hexNum);

printf("Binary equivalent: "); hexToBinary(hexNum);

printf("\n");

return 0;
}
```

Experiment 2

Program to find the 2’s complement of a given binary number

```
#include <stdio.h> #include <string.h>

int main()
{
char binary[100], onesComp[100], twosComp[100]; int i, len, carry = 1;
printf("Enter a binary number: "); scanf("%s", binary);
len = strlen(binary);

// 1's Complement
for (i = 0; i < len; i++)
{
if (binary[i] == '0')
onesComp[i] = '1'; else
onesComp[i] = '0';
}
onesComp[len] = '\0';

// 2's Complement
strcpy(twosComp, onesComp);
for (i = len - 1; i >= 0; i--)
{
if (carry == 1)
{
if (twosComp[i] == '1') twosComp[i] = '0';
else
{
twosComp[i] = '1';
carry = 0;
}
}
}

printf("\nOriginal: %s\n", binary); printf("1's Complement:%s\n", onesComp);
    printf("2's Complement:%s\n", twosComp);

return 0;
}
```

Experiment 3

Arithmetic Operations in 8086

```
org 100h
MOV AL,6
MOV BL,3
ADD AL, BL
MOV AL,9
MOV BL,3
SUB AL, BL
MOV AL,5
MOV BL,2
IMUL BL
MOV AL,4
MOV BL,2
DIV BL
ret
```

Experiment 4

Basic logical operations in 8086

```
    ORG 10H
    MOV AX, CS
    MOV DS, AX

    MOV AL, 35H
    AND AL, 5AH
    MOV BL, AL

    MOV AL, 35H
    OR  AL, 5AH
    MOV CL, AL

    MOV AL, 35H
    XOR AL, 5AH
    MOV DL, AL

    MOV AL, 35H
    NOT AL
    MOV DH, AL

    MOV AH, 4CH
    INT 21H

END
```

Experiment 5

Multiplication of two signed 8-bit numbers using Booth’s Algorithm

---

```
#include <stdio.h>
int main()
{
    int multiplicand, multiplier;
    int product;
    printf("Enter multiplicand: ");
    scanf("%d", &multiplicand);
    printf("Enter multiplier: ");
    scanf("%d", &multiplier);
    // Booth's Algorithm
    int A = 0;
    int Q = multiplier;
    int M = multiplicand;
    int Q_1 = 0;
    int n = 4;

    for (int i = 0; i < n; i++)
    {
        int Q0 = Q & 1;

        if (Q0 == 1 && Q_1 == 0)
            A = A - M;
        else if (Q0 == 0 && Q_1 == 1)
            A = A + M;

        // Arithmetic right shift
        Q_1 = Q & 1;
        Q = (Q >> 1) | ((A & 1) << (n - 1));
        A = A >> 1;
    }

    product = (A << n) | (Q & ((1 << n) - 1));

    printf("Product = %d\n", product);

    return 0;
}
```

Experiment 6

To write and execute C program to perform unsigned division of two 8-bit numbers using the Restoring Division Algorithm.

```
#include <stdio.h>

int main()
{
    int A = 0;          // Accumulator
    int Q, M;           // Quotient and Divisor
    int n = 8;          // Number of bits

    printf("Enter Dividend: ");
    scanf("%d", &Q);

    printf("Enter Divisor: ");
    scanf("%d", &M);

    // Restoring Division
    for(int i = 0; i < n; i++)
    {
        // Left shift [A,Q]
        A = (A << 1) | ((Q >> 7) & 1);
        Q = (Q << 1) & 255;

        // Subtract M from A
        A = A - M;

        if(A < 0)
        {
            // Restore A
            A = A + M;

            // Q0 = 0
            Q = Q & 254;
        }
        else
        {
            // Q0 = 1
            Q = Q | 1;
        }
    }

    printf("\nQuotient = %d", Q);
    printf("\nRemainder = %d", A);

    return 0;
}
```

Experiment 7

Write Assembly Language Program for drawing square using INT 10H.

```
org 100h
MOV AX,0012H
INT 10H
MOV BX,60H
MOV CX,60H
MOV DX,60H
MOV AL,02H
L1: MOV AH,0CH
 INC CX
 DEC BX
 INT 10H
 JNZ L1
MOV BX,60H
MOV AL,02H
L2: MOV AH,0CH
 INC DX
 DEC BX
 INT 10H
 JNZ L2
MOV BX,60H
MOV AL,02H
L3: MOV AH,0CH
 DEC CX
 DEC BX
 INT 10H
 JNZ L3
MOV BX,60H
MOV AL,02H
L4: MOV AH,0CH
 DEC DX
 DEC BX
 INT 10H
 JNZ L4
ret
```

Experiment 8

Write 8086 assembly language program for performing sorting of array.

```
ORG 100H
LEA SI, ARR
MOV CX, N-1
OUTER_LOOP:
 PUSH CX
 LEA SI, ARR
 MOV CX, N-1
INNER_LOOP:
 MOV AL, [SI]
 MOV AH, [SI+1]
 CMP AL, AH
 JBE NO_SWAP
 MOV [SI], AH
 MOV [SI+1], AL
NO_SWAP:
 INC SI
 LOOP INNER_LOOP
 POP CX
 LOOP OUTER_LOOP
; ---------- PRINT SORTED ARRAY ----------
LEA SI, ARR
MOV CX, N
PRINT_LOOP:
 MOV DL, [SI]
 ADD DL, '0' ; convert digit to ASCII
 MOV AH, 02H
 INT 21H
 MOV DL, ' '
 MOV AH, 02H
 INT 21H
 INC SI
 LOOP PRINT_LOOP
 MOV AH, 4CH

 INT 21H
ARR DB 5, 2, 9, 1, 7, 6, 3, 8
N EQU 8
END
ret
```

Experiment 9

Block transfer of data using index registers (SI and DI)

```
.MODEL SMALL
.DATA
SRC DB 11H,22H,33H,44H,55H   ; source block
DST DB 5 DUP(0)              ; destination block
.CODE
MAIN:
    MOV AX,@DATA
    MOV DS,AX                ; initialize DS
    MOV ES,AX                ; initialize ES (ES = DS)
    LEA SI,SRC               ; SI -> source block
    LEA DI,DST               ; DI -> destination block
    MOV CX,05H               ; number of bytes to transfer
    CLD                      ; DF = 0 (auto-increment SI, DI)
    REP MOVSB                ; copy CX bytes from DS:SI to ES:DI
    MOV AH,4CH
    INT 21H                  ; exit to DOS
END MAIN
```

Experiment 10

Display of uppercase and lowercase alphabets using INT 21H

```
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN:
    MOV AX,@DATA
    MOV DS,AX            ; initialize data segment

    MOV DL,'A'           ; DL = 41H (ASCII of 'A')
    MOV CX,26            ; 26 uppercase letters
UPPER:
    MOV AH,02H           ; display character function
    INT 21H              ; print character in DL
    INC DL               ; next letter
    LOOP UPPER

    MOV AH,02H           ; new line (CR + LF)
    MOV DL,0DH
    INT 21H
    MOV DL,0AH
    INT 21H

    MOV DL,'a'           ; DL = 61H (ASCII of 'a')
    MOV CX,26            ; 26 lowercase letters
LOWER:
    MOV AH,02H
    INT 21H
    INC DL
    LOOP LOWER

    MOV AH,4CH
    INT 21H              ; exit to DOS
END MAIN
```